Chapter 14 -- Semantic Requirements: Existential Restriction and the Logic of "Some"

> **Editor's note:** This chapter is Part 1 of a three-chapter unit on OWL property restrictions (originally drafted as a single Chapter 14). Chapter 14 covers *why* restrictions matter and the existential restriction (`some`). Chapter 15 covers the universal restriction (`only`), their combination, and a Neo4j implementation walkthrough. Chapter 16 synthesizes both into the EKA framework, reviews common mistakes, and closes the unit.

Table of Contents:

- 14.1 Chapter Introduction -- From Semantic Boundaries to Semantic Requirements
- 14.2 Why RDF and RDFS Are Not Enough
- 14.3 Property Restrictions -- Introducing Semantic Logic in OWL
- 14.4 Quantifier Restriction -- Understanding Existential and Universal Logic
  - 14.4.1 Existential Restriction -- "At Least One Exist"
    - === Interesting Read: More mathematical notation ===
  - 14.4.2 Universal Restriction -- "Only These Are Allowed"
    - === Interesting Read: More mathematical notation ===
  - 14.4.3 Why Chapter 14 Begins with Existential Restriction
  - 14.4.4 OWL Restriction Syntax Pattern -- Understanding the Grammar of Semantic Constraints
- 14.5 Existential Restriction in `Pizza.owl` -- Understanding Tutorial 4.10.1 and 4.10.2
  - 14.5.1 A Detail Look at Existential Restriction
  - 14.5.2 From Semantic Requirements to Executable Queries: The EKA Knowledge Graph Bridge
  - 14.5.3 Visualized Examine on Tutorial 4.10.1 and 4.10.2
  - 14.5.4 View from Open World Assumption (OWA)
- 14.6 Exercise 13 Walkthrough -- Creating Semantic Requirements in Protégé
- 14.7 Understanding Reasoning Outcomes -- Asserted vs. Inferred Semantics
  - 14.7.1 Asserted Semantics -- Explicitly Modeled Knowledge
  - 14.7.2 Inferred Semantics -- Knowledge Derived Through Logic
    - === Interesting Read: Necessary vs. Sufficient Conditions ===
  - 14.7.3 Why Existential Restriction Enables Inference
  - 14.7.4 Reasoners Think Logically -- Not Intuitively
- 14.8 Chapter 14 -> 15 Bridge
- 14.9 Key Concepts (Chapter 14)
- 14.10 Protégé Skills Learned (Chapter 14)
- 14.11 Next Chapter (15) Preview

14.1 Chapter Introduction -- From Semantic Boundaries to Semantic Requirements

By the end of Chapter (13), you had already begun understanding an important truth about ontology engineering:

> semantic relationships alone are not sufficient to fully describe meaning!

Through **Property Domain and Range**, ontology engineers learned how to establish:

> semantic boundaries.

For example, `hasTopping` may define:

Domain	Range
`Pizza`	`PizzaTopping`

This tells ontology something important:

> pizzas may have toppings.

Likewise:

> topping belong to pizzas.

However, a subtle but important limitation still remains.

Consider the following question:

> Does every pizza necessarily have toppings?

From the perspective of:

> Domain and Range alone,

the answer is:

> **not necessarily.**

Why?

Because Domain and Range describe:

> where relationships are logically valid,

but they do not express:

> what relationships are semantically required.

This distinction represents one of the most important maturity transitions in ontology engineering.

In earlier chapters, ontology primarily focused on:

> semantic structure.

You created:

- classes
- subclass hierarchies
- object properties
- inverse properties
- property characteristics
- semantic boundaries

Ontology therefore answered questions such as:

- What things exist?
- How are concepts related?
- What semantic relationships are valid?

Chapter 14 now introduces a deeper semantic question:

> **What relationships must exist?**

This shift is profound.

Because ontology is no longer merely describing:

> allowable semantic connections.

Ontology now begins defining:

> **semantic obligations.**

For example:

Saying:

> Pizza `hasBase` PizzaBase

describes:

> a valid relationship.

But saying:

> every Pizza must have at least one PizzaBase

introduces something fundamentally different:

> a semantic requirement.

Ontology therefore begins evolving from:

> connected semantics

toward:

> **logical semantics.**

This chapter begins with one of OWL's most powerful modeling constructs:

> **Existential Restriction**

often expressed through:

> `someValuesFrom`

At first glance, existential restrictions may appear technically simple.

Yet conceptually, they represent a major milestone!

Because this is the point where ontology starts moving toward:

> **machine-understandable logic.**

Rather than merely storing knowledge, ontology begins expressing:

> how knowledge should behave.

Within the broader direction of:

> **Executable Knowledge Architecture (EKA)**

this chapter represents another maturity leap:

from:

> governed semantic relationships

toward:

> **governed semantic requirements.**

This is also where the industry slogan *Meaning before Automation* -- associated with Frédéric Verhelst's *The Ontology Imperative* -- becomes concrete at the smallest possible scale. Existential restriction is the first place in the `Pizza` ontology where meaning is agreed before anything is automated: **the class definition states what must be true, and only then does the reasoner act on it**.

> **Within this chapter, using `someValuesFrom` is interchangeable with `some`; `allValuesFrom` is interchangeable with `only`.**

14.2 Why RDF and RDFS Are Not Enough

To fully appreciate why `existential restrictions` matter, it is helpful to revisit a theme introduced earlier in:

> **Chapter 08 -- RDF as a Language**

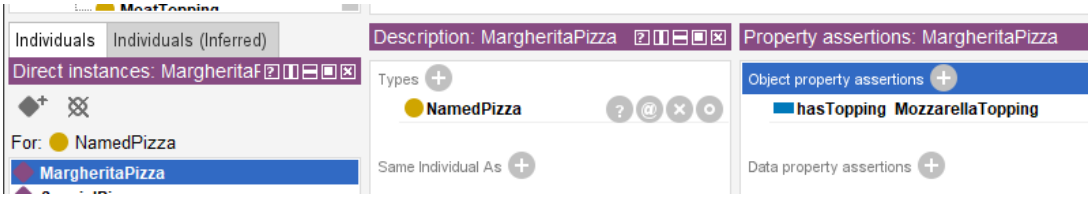
As discussed previously, RDF provides a remarkably elegant mechanism for representing knowledge.

Through simple triple "subject → predicate → object", RDF allows semantic relationships to be represented consistently.

For example:

> MargheritaPizza → `hasTopping` → MozzarellaTopping

In Protégé, we model `Pizza.owl` as below screen:



This structure gives ontology an important foundation:

> connected meaning.

Later, RDFS (RDF Schema) extends this capability by introducing:

- class hierarchies
- subclass relationships
- Domain definitions
- Range definitions

These additions improve semantic understanding considerably.

For example:

RDFS can express:

> Pizza is a subclass of Food

or:

> `hasTopping` connects "Pizza → PizzaTopping".

At this stage, ontology becomes:

> semantically structured.

However, an important limitation still exists!

RDFS can describe:

> semantic possibility.

But it cannot adequately express:

> semantic **necessity**!

This distinction becomes critically important.

Consider the following statement:

> Pizza `hasBase` PizzaBase

What does this actually mean?

It tells ontology:

> pizza **may** logically has pizza bases.

But does ontology know that:

> every pizza **must** have a base?

NO!

RDFS alone cannot fully express this requirement.

Likewise:

> MargheritaPizza `hasTopping` MozzarellaTopping

does not automatically mean:

> every MargheritaPizza requires mozzarella.

Ontology still lacks a way to formalize:

> semantic **obligation**.

This limitation becomes increasingly problematic in real enterprise modeling.

Imagine an enterprise architecture ontology.

Suppose an organization defines:

> Application `hostedOn` Infrastructure

This relationship may be semantically valid.

However, enterprise architects often need stronger semantic meaning, like:

> every production application must be `hosted on` infrastructure.

Notice the difference.

The first statement describes: **possibility**;

The second introduces: **requirement**.

This is precisely where:

> **OWL (Web Ontology Language)**

extends semantic capability beyond RDF and RDFS.

OWL introduces:

> logical expressiveness.

Ontology now becomes capable of describing:

- Rules,
- Requirements,
- Obligations, and
- **machine-interpretable semantic logic**.

This transition is important enough to view as a fundamental evolution:

- **RDF** - connected data - *"Something is connected to something."*

- **RDFS** - structured semantics - *"These kinds of things may be connected in these ways."*
- **OWL** - logical semantics - *"These kinds of things **must** be connected in these ways."*

14.3 Property Restrictions -- Introducing Semantic Logic in OWL

By the end of Chapter (13), ontology had already become significantly more expressive.

You could now model:

- classes and subclass hierarchies
- object properties and inverse properties
- property characteristics
- semantic boundaries through Domain and Range

However, ontology skill lacked an important capability:

> **the ability to formally describe semantic conditions.**

Consider the following statement:

```
Pizza  hasTopping  PizzaTopping
```

This tells ontology something useful already:

> pizza may have toppings.

Yet, ontology still cannot answer a deeper semantic question:

> what makes a pizza become a specific kind of pizza?

For example:

What semantically distinguishes `MargheritaPizza` from `AmericanPizza` or `VegetarianPizza`?

Either pizza above may have some topping, but merely defining classes and relationships like this way is insufficient.

Ontology now requires a mechanism capable of expression:

> **semantic constraints**

and:

> **logical requirements.**

This need introduces one of the most important constructs within:

> **OWL (Web Ontology Language)**

knows as:

> **Property Restrictions.**

Property restrictions allow ontology engineers to formally describe:

- how properties should behave?
- what relationships are expected? and
- what semantic conditions must hold?

In other words, property restrictions move ontology from:

> semantic structure

toward:

> **semantic logic.**

This distinction is fundamental.

Because until now, ontology primarily described:

> what thinks exist

and

> how things connect.

With property restrictions, ontology begins expressing:

> **what things mean.**

This is one of the first moments where ontology becomes **logically interpretable**.

And consequently **reasoner-friendly.**

Within OWL, property restrictions are represented as:

> logical class descriptions.

Meaning: A class can now be defined not merely through **a name**, but through **semantic conditions.**

For example:

Instead of manually declaring:

> `CheesyPizza`

ontology can describe it logically as:

> a `Pizza` that has **at least one** `CheeseTopping` .

This subtle shift changes ontology dramatically.

Meaning is no longer **manually assigned.**

Meaning becomes **logically inferable.**

To better understand property restrictions, it is useful to view them as a family of semantic mechanisms.

```
./
├─ Quantifier Restriction
│  ├─ Existential Restrictions
│  └─ Universal Restrictions
├─ Cardinality Restrictions
└─ hasValue Restrictions
```

Broadly speaking as above tree-view, OWL property restrictions can be grouped into three major categories:

1. Quantifier Restrictions

Quantifier restrictions describes:

> **whether certain relationships exist**

or:

> whether relationships are restricted to certain types.

These restrictions focus on semantic existence and semantic scope.

Quantifier restrictions include:

Existential Restriction

(`someValuesFrom`)

Meaning: there exists at least one relationship satisfying a condition.

Example:

> `Pizza` `hasTopping` some `CheeseTopping`

Meaning:

> every pizza must have at least one cheese topping.

Universal Restriction

(`allValuesFrom`)

Meaning: all (every) successor individual must belongs to a specified class.

Example:

> `Pizza` `hasTopping` only `PizzaTopping`

Meaning:

> every topping of a pizza must be a `PizzaTopping` .

Notice the important distinction.

- Existential restriction expresses: **minimum semantic existence.**
- Universal restriction expresses: **semantic limitation.**

These two concepts are frequently confused by beginners.

Yet they represent fundamentally different forms of semantic reasoning.

Chapter (14) begins with:

> **Existential Restriction**

because ontology must first understand:

> required existence

before learning:

> restricted universality.

Do you see any similarity for this ontology (machine) learning approach with how human being learn? True, they are in the common approach.

2. Cardinality Restrictions

While quantifier restrictions focus on existence, cardinality restrictions focus on **quantity.**

Ontology may sometimes need to define:

> how many relationships are permitted or required.

OWL therefore supports several forms of:

> cardinality constraints.

Including:

(1) Minimum Cardinality

(`minCardinality`)

Meaning: at least N relationships must exist.

Example:

> `Pizza` `hasTopping` minimum 2

Meaning:

> a pizza must have at least two toppings

(2) Maximum Cardinality

(`maxCardinality`)

Meaning: no more than N relationships may exist.

Example:

> `Person` `hasBiologicalMother` maximum 1

Meaning:

> a person cannot have more than one biological mother.

(3) Exact Cardinality

(`exactCardinality`)

Meaning: exactly N relationships are required.

Example:

> `Standard_Bicycle` `hasWheel` exactly 2

Meaning:

> standard bicycles must have precisely two wheels.

Cardinality restrictions become particularly important in:

- enterprise governance,
- data quality validation, and
- business rule enforcement.

Within enterprise architecture, examples may include:

> an `business_application` must have exactly one `system_owner`

or:

> a `business_process` must involve at least one `responsible_role` .

Ontology therefore becomes increasingly capable of expressing:

> operational logic,

rather than merely:

> descriptive semantics.

3. Value Restrictions

The third major category involves **specific required values** represented through `hasValue`.

Unlike existential restriction, which asks for:

> as least one qualifying relationship,

value restriction specifies:

> a particular exact value.

For example:

Suppose an enterprise policy states:

> all `production_applications` must belong to environment " `Production` ".

Ontology may express:

```
hasEnvironment value Production
```

This means:

> the relationship must point to a specific predefined individual.

In `Pizza.owl` terms, imagine requiring:

> a `NamedPizza` must always have a particular base.

Rather than merely saying:

> some pizza base exists,

ontology would specify:

> one exact expected value.

Value restrictions therefore introduce:

> semantic precision

as well as:

> semantic determinism.

Together, these three categories establish the foundation of:

> OWL semantic logic.

They transform ontology from:

> semantic modeling

into:

> formal semantic definition.

Summarized view in below comparison table from also the evolution discussed in 14.2 from RDF/RDFS to OWL:

Layer	Core Capability	Limitation
RDF	Triples (connected meaning)	No structure
RDFS	Class hierarchies, Domain & Range (structured semantics)	Can only describe **possibility**, cannot express **necessity**
OWL	Logical expressiveness (rules, requirements, obligations)	--

Existential restrictions represent one of the earliest moments where you may begin seeing:

> ontology as **logic**.

Rather than merely:

> ontology as structure.

And this transition fundamentally changes how ontology can support:

- Reasoning (R),
- Governance (Γ), and eventually
- executable intelligence.

Among these restriction categories, **quantifier restrictions** are typically introduced first because they provide the conceptual foundation for semantic participation and logical class definition. We therefore begin with existential and universal restrictions.

14.4 Quantifier Restriction -- Understanding Existential and Universal Logic

Among all OWL restrictions, the most foundational are:

> **Quantifier Restrictions.**

These restrictions focus on a deceptively simple question:

> **what kind of relationships should exist?**

Quantifier restrictions allow ontology to reason about:

> semantic participation.

In other words:

ontology begins understanding not merely:

> whether concepts are connected,

but:

> how those connections should behave logically.

OWL provides two primary forms of quantifier restriction:

- **Existential Restriction**: represented by `someValuesFrom`
- **Universal Restriction**: represented by `allValuesFrom`

Although these may appear similar at first glance, their semantic meaning differs significantly.

14.4.1 Existential Restriction -- "At Least One Exist"

Existential restriction expresses:

> **there exists at least one qualifying relationship.**

In **Description Logic (DL)**, this is often represented conceptually as:

> $\exists R.C$

Meaning: there exists at least one (some) relationship R to something belonging to an individual belonging to class C.

=== Interesting Read: More mathematical notation ===

Expand above notation in DL, the concept of an existential restriction is formally defined using the following mathematical notation:

$$\exists R.C = \{x \in \Delta^{\mathcal{I}} \mid \exists y \in \Delta^{\mathcal{I}} : (x, y) \in R^{\mathcal{I}} \wedge y \in C^{\mathcal{I}}\}$$

Breakdown of this formula --

- $\Delta^{\mathcal{I}}$: The domain of interpretation, representing the set of all individuals in the ontology world.
- x, y: Individual elements within that interpretation domain.
- $R^{\mathcal{I}}$: The interpretation of the role (relationship) R, represented as a binary relation between individuals.
- $C^{\mathcal{I}}$: The interpretation of the concept (class) C, represented as the set of all individuals belonging to class C.
- $(x, y) \in R^{\mathcal{I}}$: Indicates that a relationship R exists between individual x and individual y.
- $y \in C^{\mathcal{I}}$: Indicates that individual y belongs to class C.
- $\wedge$: Logical conjunction (AND), meaning both conditions must hold simultaneously.

Conceptual visualization --

To understand this mapping, let's imaging a `parentTo` relationship connected to a `Person` class: $\exists parentTo.Person$

Meaning:

> there exists at least one child connected through `parentTo` to a `Person`.

In short, the expression $\exists R.C$ describes the set of all individuals x that have **at least one successor** y through relationship R, such that y belongs to class C.

=== END ===

Within our `Pizza.owl` ontology, consider:

```
hasTopping some MozzarellaTopping
```

Semantically, this means:

> there exists at least one topping relationship to mozzarella topping.

This statement introduces something very important:

> semantic necessity.

Ontology now expects:

> at least one qualifying relationship.

This differs significantly from earlier chapters.

- Previously: relationship were optional.
- Now: relationships become logically meaningful (necessity).

A useful way to understand existential restriction is through the phrase:

> **AT LEAST ONE.**

Not:

> exactly one.

Not:

> all toppings.

Simply: there exists at least one!

This distinction matters enormously.

Because ontology reasoning depends heavily on:

> precise semantics.

Let's see an example in `Pizza.owl`:

Suppose a class defines:

> `MargheritaPizza`

as:

> `hasTopping some MozzarellaTopping`

Ontology now understand:

> mozzarella topping is semantically necessary by Margherita pizza.

Without satisfying this requirement:

> the pizza cannot fully conform to the class meaning, semantically.

This introduces another important shift.

Ontology modeling now begins answering:

> **What makes something what it is?**

A `MargheritaPizza` is not merely:

> a pizza.

It becomes:

> a pizza satisfying semantic conditions.

This distinction transform ontology engineering from:

> classification

toward:

> **formal semantic definition**.

Ontology therefore becomes increasingly capable of expressing:

> adaptive domain knowledge.

In enterprise context, this becomes extremely valuable.

See another example:

A `Critical_Application` may require:

> at least one `Disaster_Recovery_Mechanism`.

Or:

`Sensitive_Data_Process` may require:

> at least one `Compliance_Control`.

Ontology can now formally describe:

> mandatory semantic conditions.

Rather than merely:

> possible relationships.

This marks another important maturity leap toward:

> executable knowledge systems.

14.4.2 Universal Restriction -- "Only These Are Allowed"

Universal restriction expresses:

> **all successor individuals must satisfy a condition.**

Conceptually:

> $\forall R.C$

Meaning: every relationship R must point to class C.

=== Interesting Read: More mathematical notation ===

Expand above notation in DL, the concept of a universal restriction is formally defined using the following mathematical notation:

$$\forall R.C = \{x \in \Delta^{\mathcal{I}} \mid \forall y \in \Delta^{\mathcal{I}} : (x, y) \in R^{\mathcal{I}} \rightarrow y \in C^{\mathcal{I}}\}$$

Breakdown of this formula --

- $\Delta^{\mathcal{I}}$: The domain of interpretation, representing the set of all individuals in the ontology world.
- x, y: Individual elements within that interpretation domain.
- $R^{\mathcal{I}}$: The interpretation of the role (relationship) R, represented as a binary relation between individuals.
- $C^{\mathcal{I}}$: The interpretation of the concept (class) C, represented as the set of all individuals belonging to class C.
- $(x, y) \in R^{\mathcal{I}}$: Indicates that a relationship R exists between individual x and individual y.
- $y \in C^{\mathcal{I}}$: Indicates that individual y belongs to class C.
- $\rightarrow$: Logical implication (IF…THEN), meaning if relationship R exists, the target individual must satisfy class condition C.

Conceptual visualization --

To understand this semantic interpretation, same as previous example, imagine a `parentTo` relationship connected to `Person`, now as $\forall parentTo.Person$

Meaning:

> all individuals connected through `parentTo` must belong to the class `Person`.

Importantly, this does **not** imply that a person necessarily satisfies `parentTo` relationship.

Instead, ontology expresses:

> if a `parentTo` relationship exists, it must only point to valid `Person` individuals.

In short, the expression $\forall R.C$ describes the set of all individuals x such that:

> every successor individual y connected through relationship R must belong to class C.

This is why universal restriction expresses:

> **semantic limitation**

rather than:

> semantic existence.

In short, the expression $\forall R.C$ describes the set of all individuals x whose **every successor** y through relationship R must belong to class C.

=== END ===

For example:

> `Pizza` `hasTopping` only `PizzaTopping`

Meaning:

> all toppings of a pizza must belong to PizzaTopping.

This does **not** guarantee toppings exist.

Instead, it governs:

> semantic limitation.

This distinction is critically important.

Because beginners often mistakenly assume `only` means **required**.

But semantically, it means **constrained**.

A pizza may still have:

> zero toppings.

Ontology here simply says:

> if topping exist,

they must belong to:

> `PizzaTopping` .

Understanding this distinction is one of the earliest maturity moments in ontology engineering.

Because ontology logic depends heavily on:

> semantic precision.

14.4.3 Why Chapter 14 Begins with Existential Restriction

Michael intentionally introduces:

> existential restriction

before universal restriction.

This teaching sequence is important.

Because ontology first needs to understand:

> **semantic existence**

before introducing:

> **semantic limitation**

In practical modeling:

it is usually easier to first ask:

> what relationships are required?

before asking:

> what relationships are allowed?

This chapter therefore focuses on:

> **Existential Restriction**

using:

> `someValuesFrom`

before later chapters gradually expand toward:

> more advanced logical constraints.

14.4.4 OWL Restriction Syntax Pattern -- Understanding the Grammar of Semantic Constraints

After understanding the conceptual difference between:

> existential restriction (`someValuesFrom`)

and:

> universal restriction (`allValuesFrom`),

you should pause briefly to recognize an important fact, from language perspective, that:

> OWL restriction are not random syntax.

Instead, they follow a highly structured semantic grammar.

This is an important transition point in ontology learning.

Because many beginners mistakenly perceive expressions such as:

`hasTopping some MozzarellaTopping`

as:

> Protégé-specific notation

or simply:

> a software configuration format.

However, this interpretation is incomplete.

In reality, OWL restrictions represent:

> **formal semantic expressions**

which are used to describe:

> logical class conditions.

In other words:

OWL restriction statements behave more like:

> **semantic sentences**

than:

> user interface settings.

Understanding this grammar is important because future ontology modeling will increasingly depend upon:

> composing semantic logic

rather than merely:

> creating classes and relationships.

At a further high level, most OWL restriction expressions follow a common structural pattern:

$$ObjectProperty + RestrictionType + TargetClass$$

Conceptually:

$$Relationship + SemanticLogic + SemanticTarget$$

This structure can be understood as a reusable language pattern for constructing:

> **semantic constraints.**

Let us examine a common example from `Pizza.owl`:

`hasTopping some MozzarellaTopping`

This restriction can be decomposed into three semantic components:

Component	Meaning	Semantic Role
`hasTopping`	Object Property	Defines the relationship
`some`	Restriction Type	Defines existential logic
`MozzarellaTopping`	Target Class	Defines the semantic requirement

When interpreted together, ontology understands the expression as:

> a pizza must have **at least one** topping belonging to the class `MozzarellaTopping`.

Very importantly: the keyword `some` does **NOT** mean "several" or "an unspecified number"!

Within OWL semantics: `some` specifically represents **existential quantification**, meaning:

> **at least one qualifying relationship exists.**

Now consider a second example:

`hasTopping only PizzaTopping`

This expression follows the exact same grammar pattern:

Component	Meaning	Semantic Role
`hasTopping`	Object Property	Defines the relationship
`only`	Restriction Type	Defines universal logic
`PizzaTopping`	Target Class	Defines the semantic requirement

However, the semantic interpretation changes significantly.

Ontology now understand:

> all toppings associated with a pizza must belong to the class `PizzaTopping`.

Notice what changed.

The **relationship** remains identical.

The **target class** also remains similar.

Only the **restriction logic** changes.

Yet this small syntactic variation completely transforms semantic meaning.

This illustrates an important principle in ontology engineering:

> **small changes in OWL syntax may produce major differences in logical interpretation.**

As ontology modeling becomes more advanced, you will encounter increasingly sophisticated restriction patterns.

For example:

- Existential Quantifier Restrictions: `hasTopping some CheeseTopping`
- Universal Quantifier Restrictions: `hasTopping only PizzaTopping`
- Cardinality Restrictions: `hasTopping min 2`
- Exact Cardinality Restrictions: `hasWheel exactly 4`
- Value Restrictions: `hasEnvironment value Production`

Although these expressions appear different, they all follow a similar semantic grammar:

$$Property + Restriction + Target$$

Understanding this pattern is an important milestone!

Because ontology engineers gradually stop reading OWL as:

> syntax

and begin reading OWL as:

> **semantic language.**

This mindset shift is crucial.

Especially for enterprise ontology and knowledge graph engineering.

Since ontology increasingly becomes not merely:

> a data model,

but:

> **a language for expressing machine-interpretable meaning.**

With this grammar foundation established, you are now ready to begin applying existential restrictions practically inside `**Pizza.owl**` and Protégé.

14.5 Existential Restriction in `Pizza.owl` -- Understanding Tutorial 4.10.1 and 4.10.2

14.5.1 A Detail Look at Existential Restriction

After introducing the conceptual foundation of property restrictions, Michael now transitions toward one of this most important practical modeling techniques in OWL:

> **Existential Restriction**

through the construct `someValuesFrom`.

At this stage in the `Pizza.owl` tutorial, ontology modeling begins changing character.

Earlier chapters largely focused on **describing relationships.**

For example:

> Pizza `hasTopping` PizzaTopping

or:

> Pizza `hasBase` PizzaBase

These statements helped ontology understand:

> semantic structure.

However, ontology still lacked the ability to describe:

> **semantic requirements.**

This distinction is subtle but important.

Because knowing:

> a pizza may have toppings

is fundamentally different from saying:

> a pizza must contain at least one topping.

Ontology therefore begins transitioning from:

> semantic possibility

toward:

> **semantic obligation.**

14.5.2 From Semantic Requirements to Executable Queries: The EKA Knowledge Graph Bridge

This transition from "possibility" to "obligation" is not merely an academic logical exercise. Within the EKA framework, it has a very concrete and powerful implication for the K **- Knowledge Graph** layer.

Recall the EKA formalization: $EKA = (K, R, \Theta, \Phi, \Gamma)$.

- **Without Existential Restrictions (K is a "Descriptive Graph"):** The knowledge graph can only answer "what is" queries. For example, you can ask: *"Which pizzas have a* `MozzarellaTopping` *?"* This is a simple traversal of asserted relationships.
- **With Existential Restrictions (K becomes an "Intelligent Graph"):** Because the ontology now contains *logical rules* about what *must* exist, the knowledge graph can answer "what should be" and "what is missing" queries.

Example in a Graph Query (e.g., Cypher or SPARQL):

You have defined that a `CheesePizza` **must** have at least one `hasTopping` relationship to a `CheeseTopping`. Your knowledge graph contains the following individuals:

1. `PizzaA` (type: `CheesePizza`) - has a relationship `hasTopping` to `CheddarTopping`.
2. `PizzaB` (type: `CheesePizza`) - has **no** `hasTopping` relationships.

=== Additional Reading, if you have Neo4j on-hand ===

Assume your Neo4j graph contains nodes labeled `Pizza` and `CheeseTopping` (Note: `CheeseTopping` could be member of parent node `PizzaTopping`), connected by a relationship `:HAS_TOPPING`.

Based on above individuals information.

Run this Query 1:

```
MATCH (p:Pizza {type: 'CheesePizza'})
RETURN p.name AS PizzaName
```

You may get result as:

PizzaName
PizzaA
PizzaB

Both pizzas are returned, even though `PizzaB` has no toppings.

Then run this Question 2:

```
MATCH (p:Pizza {type: 'CheesePizza'})
WHERE (p)-[:HAS_TOPPING]-(>:CheeseTopping)
RETURN p.name AS ValidCheesePizzaName
```

You should see the result now as:

ValidCheesePizzaName
PizzaA

Only `PizzaA` is returned. `PizzaB` is excluded because it does not satisfy the "must have at least one cheese topping" rule.

From knowledge graph perspective:

- Query 1 represents a **descriptive knowledge graph**: it simply tells you what is asserted, but it cannot distinguish between a valid `CheesePizza` (with toppings) and an invalid or incomplete one.
- Query 2 represents the **existential restriction logic** (`hasTopping some CheeseTopping`) enhancing the knowledge graph: it queries only those individuals that *satisfy the semantic requirement*, this turns your ontology's logical rules into powerful, executable data quality filters.

In an EKA-enabled systems, Query 2 would be the basis for generating trustworthy insights, while Query 1 might be used for auditing or identifying data quality issues (i.e., finding `PizzaB` to flag it for correction).

=== END ===

Without the existential restriction (i.e., using only RDFS/SHACL), both `PizzaA` and `PizzaB` are valid members of `CheesePizza`. A query would simply return both.

With the existential restriction (using OWL and a reasoner), the semantic layer within EKA (R) can first validate the graph. It would identify `PizzaB` as **incomplete** or **inconsistent** based on the logical rule you defined in the ontology.

This allows the **Executable Intelligence (Φ)** layer to trigger actions. A governance workflow could automatically flag `PizzaB` for a data quality issue, or a smart query could be written to exclude it from results.

Therefore, Chapter (14) is the pivotal moment where your ontology stops being a mere "schema" for your knowledge graph and becomes its "logical guardian." The existential restriction (`someValuesFrom`) is your first tool to enforce business rules ("a product must have a category", "a critical application must have an owner") directly within your executable knowledge architecture.

14.5.3 Visualized Examine on Tutorial 4.10.1 and 4.10.2

Michael introduces existential restriction through a very deliberate modeling progression.

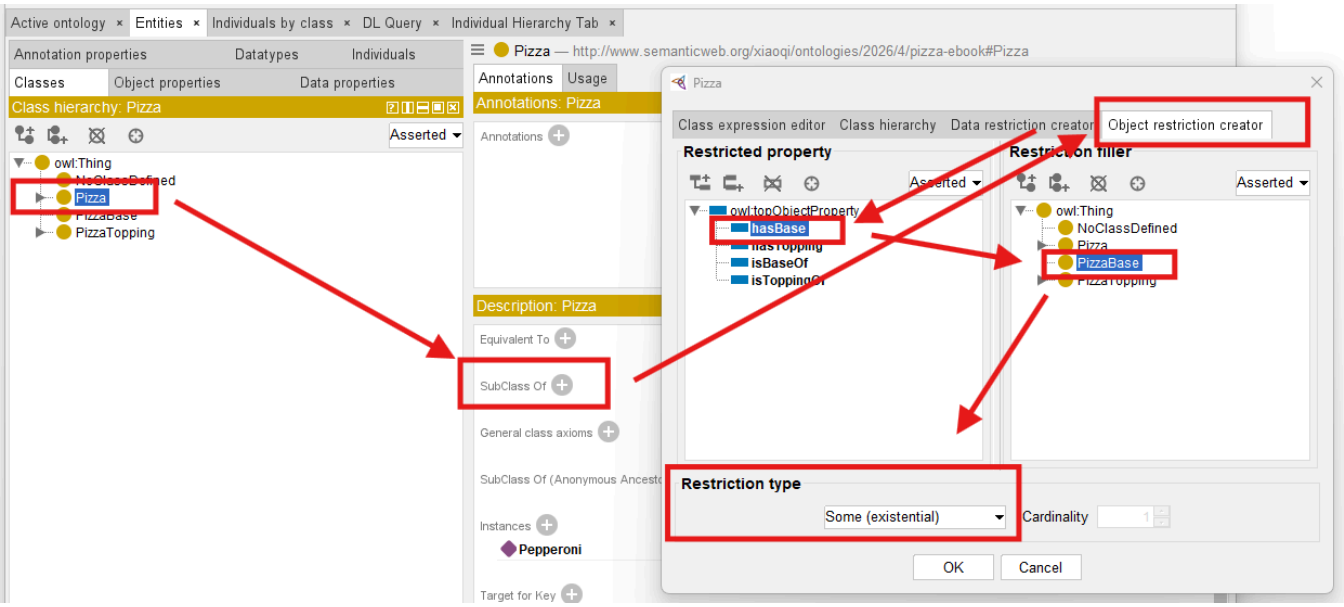
Rather than immediately creating complicated class logic, you first explore a simple but powerful idea:

> classes may be described through required relationships.

This introduces a major ontology engineering concept:

> **logical class definition.**

See below steps which described in Exercise 13 and result `Pizza hasBase some PizzaBase`:



Previously, you manually create classes primarily through naming and hierarchy.

For example:

> `MargheritaPizza`

may simply exist as:

> a subclass of `NamedPizza`.

However, ontology still does not truly understand:

> what semantically makes a `MargheritaPizza` a `MargheritaPizza`.

Existential restriction changes this.

Ontology can now begin expression:

> semantic identity.

For example:

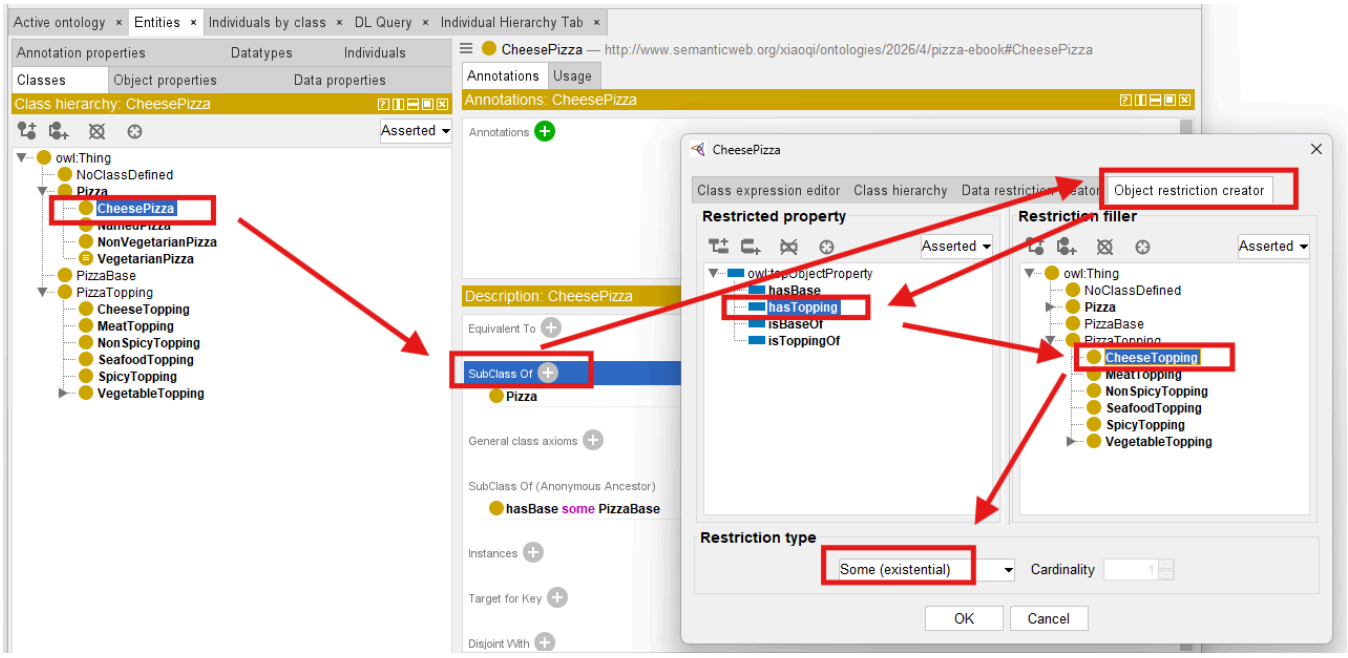
Instead of simply naming:

> `CheesePizza`,

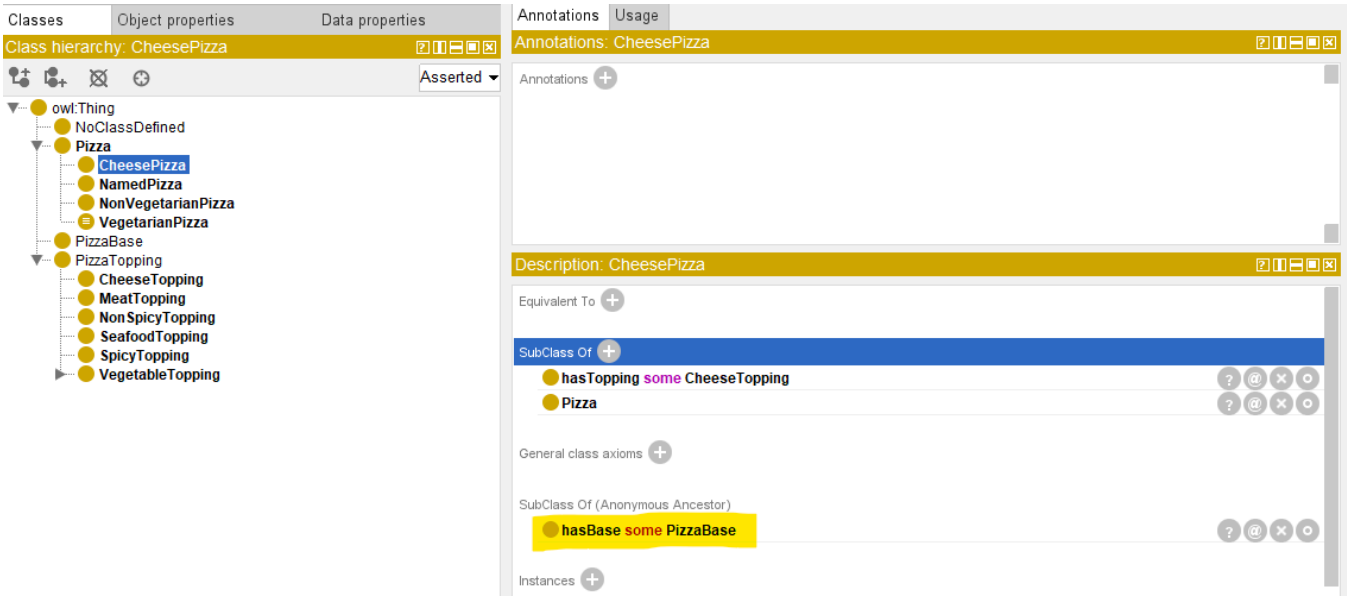
ontology may define:

> a pizza having at least one cheese topping.

See how to implement this in Protégé and resulting `CheesePizza hasTopping some CheeseTopping`:



Since `CheesePizza` is SubClass Of `Pizza`, in the result Description screen, you may see the previous Pizza / PizzaBase existential restriction is inherited, as below:



This ensure your ontology grows organically base on the known and true knowledge already have.

With such kind of configuration, your ontology is transformed fundamentally.

Meaning becomes:

> computable.

Reasoners may now determine:

> class membership automatically.

Ontology therefore becomes increasingly **inferential** rather than merely **descriptive!**

This represents one of the earliest moment where ontology begins feeling:

> intelligent.

Because class meaning no longer depends entirely upon:

> human categorization then manually configuration.

Instead:

> **logical semantics drive understanding based on machine-understandable rules.**

Within our `Pizza.owl`, existential restriction is intentionally introduced before more advanced logical restrictions because it establishes the foundational reasoning pattern:

> **required existence.**

Ontology first learns:

> something must exist.

Later chapters will teach ontology:

- what is allowed,
- what is limited, and
- how many relationships are acceptable.

This gradual semantic progression is one reason the `Pizza.owl` tutorial remains such an effective learning path.

Because ontology maturity develops incrementally.

14.5.4 View from Open World Assumption (OWA)

Interestingly, OWL reasoners do not immediately assume that a restriction has been violated simply because a required relationship has not yet been observed. In many cases, the absence of information does not necessarily imply semantic inconsistency. This behavior is closely related to one of the most important principles in ontology engineering:

> **Open World Assumption (OWA)**

which assumes that knowledge may still be incomplete.

We will explore this important concept in greater detail later in this chapter, as it fundamentally influences how OWL reasoners interpret existential restrictions and semantic requirements.

A quick example:

Suppose the ontology only knows `PizzaA` exists, with no `hasTopping` statements. Under OWA, the reasoner does not conclude `PizzaA` has no toppings. It concludes “unknown”. This is why `PizzaA` does not violate `hasTopping` `some` `CheeseTopping` — *absence of evidence is not evidence of absence.*

We will revisit OWA in detail when discussing vacuous truth (Chapter 15, section 15.1.3) and reasoner behavior (14.7.4).

14.6 Exercise 13 Walkthrough -- Creating Semantic Requirements in Protégé

With the conceptual foundation now established, you are fully ready to perform:

> Exercise 13

inside Protégé.

This exercise represents an important turning point.

For the first time, you begin configuring:

> semantic requirements

rather than merely:

> semantic structure (hierarchies and relationships).

Inside the `Pizza.owl` tutorial, you create existential restrictions using:

> `someValuesFrom`

through the Protégé interface (you already see screen samples in previous section).

At first glance, the interface interaction appears simple.

However, conceptually, something profound is happening.

Ontology is beginning to describe:

> **what a class must satisfy.**

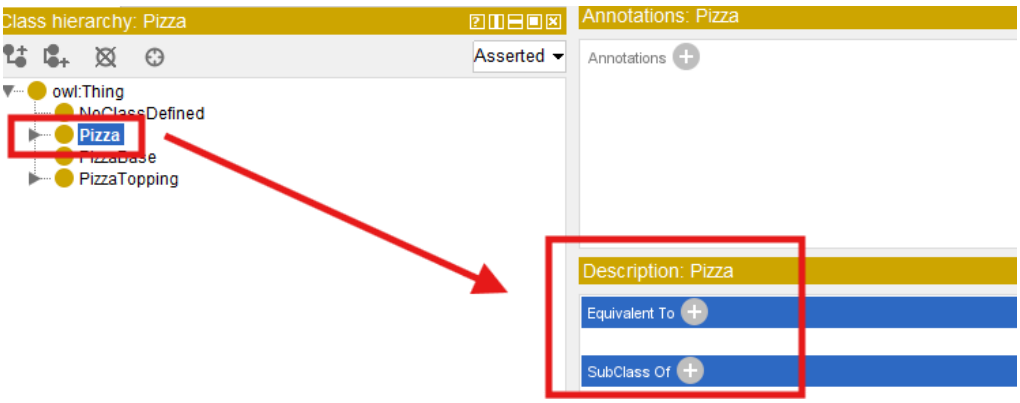
rather than merely:

> what relationships **may** exist.

Inside Protégé, you first select the target class.

Typically, Michael demonstrates this through `pizza` subclasses that require particular toppings.

You then navigates to `Equivalent Classes` or `**SubClass Restrictions**` depending on modeling preference.

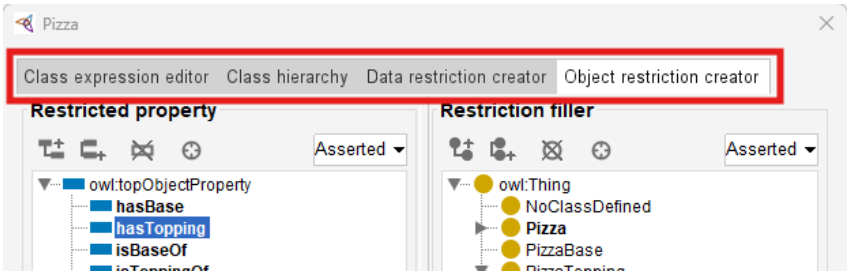


Next, Protégé provides access to:

> Object Restriction Editor.

"Editor" may not the accurate word, see below screen, you may perform more configurations here, including:

- Class expression editor
- Class hierarchy
- Data restriction creator
- Object restriction creator



At this stage, switch to actual tab name "Object restriction creator", you may perform following steps:

1. Select **Restriction Type**: `Some (existential)` means `someValuesFrom`, it's the default selection when you come to this screen, then
2. Configure **Property** in "Restricted property": `hasTopping`, and finally

3. Configure **Target Class** in "Restriction filler"

At first encounter, this may look like "Syntax".

However, ontology engineers should resist thinking about this merely as notation.

Because what has actually been created is:

> **semantic logic.**

Ontology now interprets this statement as:

> there must exist at least one topping relationship pointing toward mozzarella topping.

This changes how ontology understands "pizza identity".

A pizza now becomes classified not merely because:

> a human named it,

but because:

> semantic conditions are satisfied.

This subtle shift becomes extremely relies upon:

> inferred meaning.

rather than:

> manually asserted meaning.

=== Mathematical View ===

Let's try to map what you've configured to the previous mathematical formula:

$$\exists R.C = \{x \in \Delta^l \mid \exists y \in \Delta^l : (x, y) \in R^l \wedge y \in C^l\}$$

The statement:

> `CheesePizza` `hasTopping` some `CheeseTopping`

may have those elements referring to the variables as:

- Δ^l: the `Pizza` class domain
- R: `hasTopping` relationship
- C: `CheesePizza` class
- x, y: instance of `Pizza` (not `CheesePizza`)

A subtle but important note on x and class definitions:

> In the formula $\exists R.C = \{x \in \Delta^l \mid \exists y \in \Delta^l : (x, y) \in R^l \wedge y \in C^l\}$, the variable x ranges over instances of the class being defined. When you define `CheesePizza` as `Pizza and (hasTopping some CheeseTopping)`, the restriction is evaluated in the context of `Pizza`. This means x is an instance of `Pizza` (not necessarily an instance of `CheesePizza`). The restriction then asks: does this `Pizza` instance have at least one `hasTopping` relationship to a `CheeseTopping`? If yes, the reasoner can infer that the instance is also a `CheesePizza`. This is why x is an instance of `Pizza` in the formula – the restriction acts as a membership test for the subclass, not as a property of the subclass itself.

=== END ===

Exercise 13 therefore marks another maturity transition.

Ontology is beginning to evolve from:

> a semantic graph

toward:

> **semantic intelligence.**

14.7 Understanding Reasoning Outcomes -- Asserted vs. Inferred Semantics

After completing Exercise 13 and defining existential restrictions inside Protégé, learners may often encounter an interesting experience:

> the ontology reasoner appears to "know" things that were never manually entered.

At first glance, this may seem surprising.

Especially for readers who are more familiar with **transitional databases** or **graph databases**.

Because in those environments, data generally behaves according to a straightforward principle:

> what is **explicitly** stored is what exists.

Ontology reasoning behaves very differently.

Within OWL, semantic meaning is not limited only to:

> manually asserted facts.

Instead, reasoners are capable of producing:

> **inferred knowledge**

through logical interpretation.

This distinction represents one of the most important conceptual shifts in ontology engineering.

Because ontology gradually moves from:

> storing semantic information

toward:

> **deriving semantic meaning.**

To understand this transformation, you must first distinguish between:

Asserted Semantics and **Inferred Semantics**.

14.7.1 Asserted Semantics -- Explicitly Modeled Knowledge

Asserted semantics represent:

> information explicitly created by ontology engineers.

In other words:

there are semantic statements that humans intentionally model inside Protégé (or other ontology editors).

For example:

Suppose an ontology engineer manually creates (models):

```
PizzaA hasTopping MozzarellaTopping
```

This relationship becomes:

> asserted knowledge.

Because it was directly entered into the ontology.

Similarly:

if the ontology engineer manually states:

```
PizzaA rdf:type NamedPizza
```

this classification is likewise **asserted.**

Ontology does not need to infer it.

The semantic meaning already exists because:

> humans explicitly provided it.

From a modeling perspective, asserted semantics resemble:

> manually curated knowledge.

This is conceptually similar to:

> records stored in a database

or:

> relationships stored in a graph database.

However, ontology does not stop there.

14.7.2 Inferred Semantics -- Knowledge Derived Through Logic

Unlike asserted knowledge, inferred semantics are **not manually entered.**

Instead, they emerge through **semantic reasoning.**

> **Note**
>
> Scared? Does this mean machine starts dominating semantic creation? Terminator? Skynet?... No worry, this is still the governed logic, rooted from human's initial design!

And, this is where existential restriction becomes especially important.

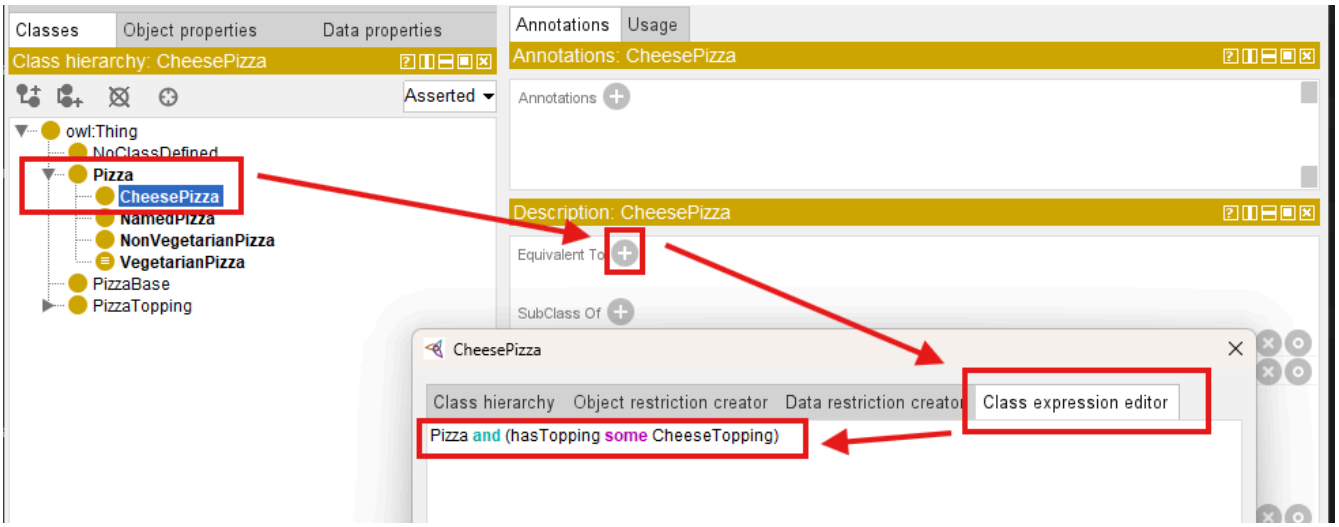
Consider the following ontology definition:

```
CheesePizza EquivalentTo
```

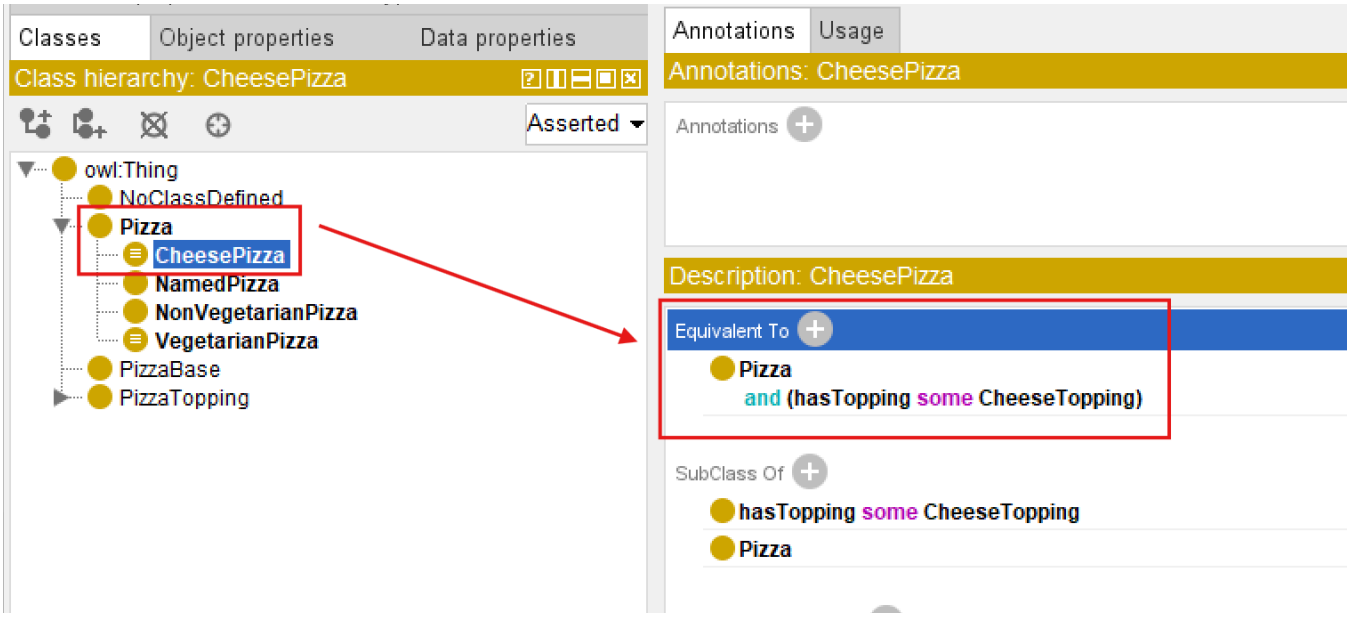
```
Pizza
```

```
and (hasTopping some CheeseTopping)
```

In Protégé, you may use below steps to add this definition:



After confirm, you may see this definition under `Equivalent To`:



This statement establishes a semantic condition.

Meaning:

> any pizza containing at least one cheese topping qualifies as a `CheesePizza`.

Note here we have the combination of more than one conditions (restrictions).

Now suppose ontology engineers explicitly model:

`PizzaA hasTopping MozzarellaTopping`

And ontology already understand:

`MozzarellaTopping SubClassOf CheeseTopping`.

Interestingly, magic happens:

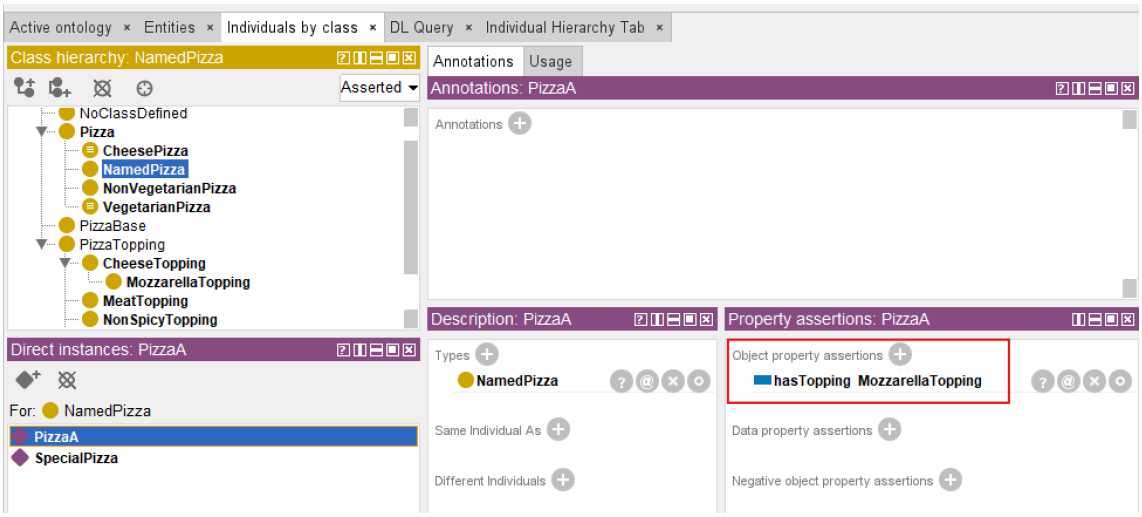
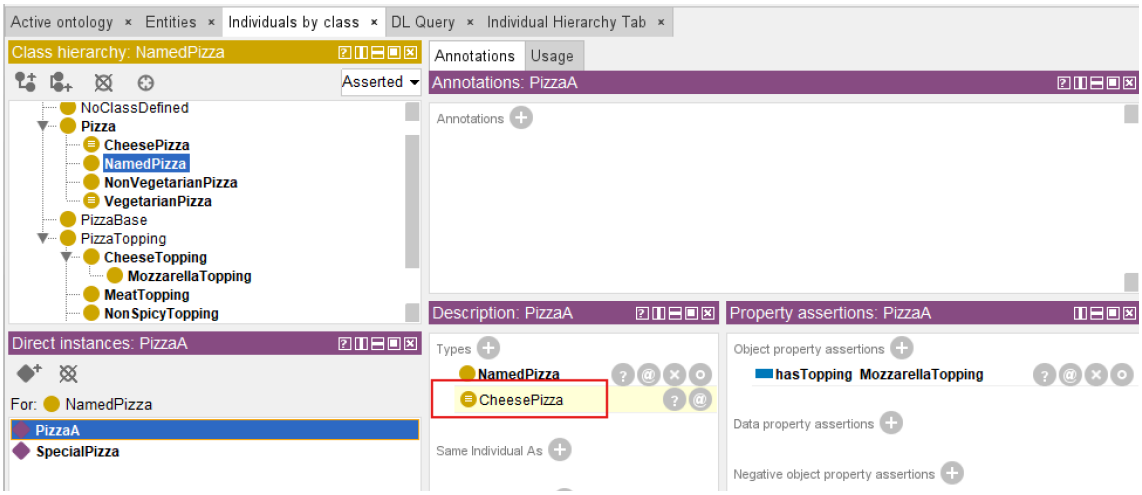
ontology engineers never explicitly declare:

`PizzaA rdf:type CheesePizza`

Yet after synchronizing the reasoner, Protégé may automatically infer:

`PizzaA rdf:type CheesePizza`

See from below comparison screen:

Before Reasoning	
After Reasoning	

This is a significant milestone!

Because ontology has now moved beyond:

> semantic storage

toward:

> **semantic interpretation.**

Meaning becomes **computable**.

The ontology reasoner analyzes:

- subclass hierarchy
- property restrictions
- existential conditions
- logical consistency, and
- **devices new knowledge**

This capability is one of the major distinctions separating ontology from:

> conventional graph data models.

Because ontology is capable of generating:

> **new semantic understanding**

rather than merely retrieving stored relationships.

=== Interesting Read: Necessary vs. Sufficient Conditions ===

The inference you just witnessed (`PizzaA` automatically classified as `CheesePizza`) depends on a fundamental logical distinction: the difference between **necessary** and **sufficient** conditions.

In Description Logic (the formal language behind OWL):

- `SubClassOf`$(\sqsubseteq)$ defines a *necessary* condition.
- `EquivalentTo`$(\equiv)$ defines *necessary and sufficient* conditions.

Mathematical form:

Concept	DL Notation	OWL Construct	Logical Direction
Necessary condition	$C \sqsubseteq D$	`SubClassOf`	$C(x) \rightarrow D(x)$
Necessary and sufficient condition	$C \equiv D$	`EquivalentTo`	$C(x) \leftrightarrow D(x)$

Example from this section:

You defined:

`CheesePizza EquivalentTo`

`Pizza`

`and (hasTopping some CheeseTopping)`

This means:

- **Necessary condition**: If something is a `CheesePizza` , it **must** have a cheese topping.
- **Sufficient condition**: If something has a cheese topping, it **can be inferred** to be a `CheesePizza` .

This is why the reasoner performed *automatic classification*. If you had used `SubClassOf` instead of `EquivalentTo` , the reasoner would **not** perform the reverse inference (from having a cheese topping to classifying as `CheesePizza`).

Why this matters for your modeling choices:

If you want...	Use...
Only inheritance (one-way)	`SubClassOf`
Logical definition with automatic classification (two-way)	`EquivalentTo`

This distinction is fundamental to ontology engineering. Choosing the wrong construct may lead to unexpected reasoning results (or missed inferences).

=== END ===

14.7.3 Why Existential Restriction Enables Inference

At this stage, you may naturally ask an important question:

> Why does existential restriction enable automatic classification?

The answer lies in the semantic meaning of the keyword:

> `some`

which represents:

> **existential quantification.**

Without OWL semantics, the expression `hasTopping some CheeseTopping` does not merely describe:

> a relationship.

Instead, it defines:

> **a logical test condition.**

More specifically, ontology interprets this expression as:

> there must exist **at least one** valid relationship through `hasTopping` pointing to an individual belonging to class `CheeseTopping` .

This subtle distinction is important.

Because ontology is not longer simply storing:

> semantic data,

but rather evaluating:

> **semantic conditions.**

In practice, ontology reasoners perform a process conceptually similar to searching for evidence.

The reasoner examines individuals inside the ontology and asks a logical question:

> "Can I find at least one relationship satisfying this existential condition?"

Suppose ontology contains:

`PizzaA hasTopping MozzarellaTopping`

and ontology already understands:

`MozzarellaTopping SubClassOf CheeseTopping`

The reasoner may then evaluate: `hasTopping some CheeseTopping` and conclude:

> YES - at least one valid successor individual exists.

This reasoning process may feel surprisingly intelligent, since it makes machine alike human.

However, semantically, the reasoner is performing something conceptually similar to:

> an automated semantic query.

Readers with graph databases experience may recognize an interesting parallel.

Inside Neo4j, a developer might write a Cypher query similar to:

```
MATCH (p:Pizza)-[:HAS_TOPPING]->(t:CheeseTopping)
RETURN p
```

or conceptually:

```
WHERE (p)-[:HAS_TOPPING]->(:CheeseTopping)
```

to locate pizzas having cheese toppings.

Ontology reasoners perform a related form of semantic evaluation.

However, instead of merely returning query results, the reasoner may additionally:

> **derive new semantic classifications.**

Meaning:

if a pizza satisfies the logical condition: `hasTopping some CheeseTopping`, the ontology reasoner may automatically conclude:

> this individual belongs to `CheesePizza` (class).

This introduces an important distinction between **graph traversal** and **semantic reasoning.**

- Graph database primarily discover **existing connections**.
- Ontology reasoners additionally derive **new meaning**!

Another important factor enabling inference is `EquivalentTo`.

Consider the following ontology definition:

```
CheesyPizza EquivalentTo

Pizza

and (hasTopping some CheeseTopping)
```

The keyword `EquivalentTo` represents:

> **necessary AND sufficient conditions**

This means the semantic rule works in **both directions.**

Ontology understands:

1. If something is a `CheesePizza`, it must satisfy the topping condition, and also
2. If something satisfies the topping condition, it may be classified as `CheesePizza`.

This bidirectional logic is extremely important.

Because it enables:

> **automatic classification.**

By comparison:

`CheesyPizza subClassOf (hasTopping some CheeseTopping)` would only define:

> a one-way semantic rule.

Meaning:

> every `CheesyPizza` must have cheese toppings,

but ontology would **not automatically infer** that:

> every pizza with cheese toppings becomes `CheesyPizza`.

This subtle modeling distinctions has major consequences for reasoning behavior.

And it highlights an important principles of ontology engineering:

> **small changes in logical constructors may produce significant different inference outcomes.**

Existential restriction therefore becomes one of the foundational mechanisms enabling ontology to evolve from:

> semantic structure

toward:

> **semantic intelligence.**

14.7.4 Reasoners Think Logically -- Not Intuitively

One of the most important mindset shifts in ontology engineering is learning that:

> ontology reasoners think logically,

not:

> **intuitively.**

From many learners, this initially feel unfamiliar and sometimes difficult to understand.

Because our humans naturally interpret missing information by making assumptions.

Ontology reasoners, however, do not (or we may also say "never") make assumption.

They follow:

> **formal logical evidence.**

This distinction becomes especially visible when working with **existential restrictions.**

Consider the following situation.

Suppose an ontology contains an individual `PizzaB` but no topping information has yet been defined.

A human reader may immediately think:

> "If no topping exists, then `PizzaB` is obviously not a `CheesyPizza`, although it has `Pizza` inside the name."

This conclusion feels natural, to humans.

After all, human reasoning frequently relies on:

> incomplete information and educated assumptions.

However, an OWL reasoner behaves differently.

The ontology reasoner does **not infer**:

> `PizzaB rdf:type CheesyPizza`

Yet importantly: the reasoner also does **not infer**:

> `PizzaB` is *not* a `CheesyPizza`.

Instead, ontology reaches a more conservative conclusion:

> **there is currently insufficient information to determine classification.**

In other words, the ontology simply says:

> **"I do not know yet."**

This behavior often surprises beginners.

Especially those with experience in

> relational databases,

> business rules engines,

or:

> graph databases.

Because many traditional systems implicitly follow a different assumption:

`Missing information = false`

Ontology reasoning, however, follows one of the most important principles in semantic modeling:

> **Open World Assumption (OWA)**

Under OWA: missing information does not imply **falsity.**

Instead, ontology assumes **knowledge may still be incomplete**.

Meaning:

> future information may still arrive, just not available for now yet.

For example:

The ontology may later receive `PizzaB hasTopping MozzarellaTopping` and ontology already understands `MozzarellaTopping SubClassOf CheeseTopping`.

At that moment, the reasoner may suddenly infer:

`PizzaB rdf:type CheesyPizza`

What has been changed?

Not the logical rule.

Only:

> **the available evidence (information).**

Ontology reasoning therefore behaves less like:

> guessing,

and more like:

> **evidence-based semantic evaluation.**

This distinction becomes clearer when comparing **Human Intuition** vs **OWL Reasoning**:

Human Intuition	OWL Reasoning Logic
No topping observed $\rightarrow$ probably not cheese pizza	No evidence observed (got) $\rightarrow$ classification postponed
Missing information suggest absence	Missing information may simply be incomplete
Infer based on assumptions	Infer based on formal evidence

Notice the key difference?

OWL reasoners do not ask "What is most likely true?"

Instead, they ask "What can be logically justified?"

This distinction becomes especially important when working with:

> existential restrictions.

Earlier in this chapter, we defined:

`CheesyPizza EquivalentTo`

`Pizza`

`and (hasTopping some CheeseTopping)`

The keyword `EquivalentTo` defines:

> **necessary and sufficient conditions.**

Meaning:

> ontology interprets the definition in **both directions.**

This enables the reasoner to conclude:

> if a pizza satisfies the existential condition, then it may be classified as `CheesyPizza` .

However, imagine the ontology instead used:

`CheesyPizza SubClassOf (hasTopping some CheeseTopping)`

This semantic meaning changes significantly.

Now ontology only understands:

> every `CheesyPizza` must satisfy the topping restriction.

But ontology can no longer conclude:

> every pizza satisfying the restriction automatically becomes `CheesyPizza` .

The inference direction becomes **one-way only.**

This subtle distinction between `EquivalentTo` and `SubClassOf` has major consequences for ontology behavior.

It also highlights an important engineering lesson:

> ontology reasoners execute logic exactly as modeled

while NOT:

> as human intended.

For ontology engineers, this carries important practical implications.

Semantic models must be designed **precisely** with careful attentions to:

- logical meaning,
- restriction semantics, and
- inference consequences.

At the same time, Open World Assumption provides an important advantage.

Because enterprise knowledge is often:

- incomplete,
- distributed, or
- continuously evolving.

OWL therefore remains highly suitable for enterprise knowledge graphs, semantic integration, and executable intelligence systems when meaning must gradually emerge from **accumulating knowledge** rather than requiring perfect information from the beginning.

As you continue through this chapter, Open World Assumption will become increasingly important for understanding:

> why ontology reasoners sometimes appear surprisingly conservative,

yet remain remarkably powerful in semantic interpretation.

14.8 Chapter 14 -> 15 Bridge

You have now built the first half of the semantic requirements toolkit: you know *why* RDF/RDFS alone cannot express requirements, you know the grammar of OWL restrictions, and you can create and reason over **existential restrictions** (`some`) -- the constraint that says "at least one relationship of this kind must exist".

But `some` only tells the reasoner what is *permitted* to exist. It says nothing about what is *forbidden*. A pizza declared to have `hasTopping some VegetableTopping` is only required to have at least one vegetable topping -- nothing stops the reasoner from also accepting a pepperoni topping -- a kind of meat topping -- on the same pizza. That gap is exactly where Chapter 15 begins, with the **universal restriction** (`only`) and the classic combination pattern that finally gives us a working definition of `VegetarianPizza EquivalentTo Pizza and (hasTopping only (CheeseTopping or VegetableTopping))` .

Put in the terms of the slogan, Chapter 14 has established the **"meaning" half** of *Meaning before Automation* for one restriction type; Chapter 15 adds the **boundary half**, and Chapter 16 shows why the pair is a governance contract rather than a modeling convenience.

14.9 Key Concepts (Chapter 14)

Concept	Description
Property Restriction	An OWL class expression that constrains how a class relates to other classes or individuals through a property.
Existential Restriction (`some`)	Requires that *at least one* relationship of a given property/class combination exists.
Asserted Semantics	Knowledge explicitly stated by the ontology engineer.
Inferred Semantics	Knowledge derived by a reasoner through logical entailment, not explicitly written.
Open World Assumption (OWA)	The reasoning stance in which absence of a statement means "unknown," not "false."

14.10 Protégé Skills Learned (Chapter 14)

- Reading and writing OWL restriction syntax (`some` , quantifier grammar)
- Creating an existential restriction on a class in Protégé
- Running Exercise 13 to attach semantic requirements to `Pizza.owl`
- Distinguishing asserted axioms from reasoner-inferred axioms in the Protégé interface

14.11 Next Chapter (15) Preview

Chapter 15 introduces the **universal restriction** (`only`), shows why `some` and `only` must be combined to correctly define `VegetarianPizza` , and walks through implementing both restriction types as Cypher constraints in Neo4j -- giving you a concrete side-by-side comparison between OWL's Open World reasoning and a property graph's Closed World queries.

Last updated at: 2026-11-12